Freefly API

Version 1.0

Revision E | 04.11.2023

Overview

The Freefly API is a serial protocol interface that allows control and status of Freefly's MōVI Pro gimbal, through the logic-level UART COM ports on the MōVI Pro Gimbal Control Unit (GCU) and the MIMIC. The core API code distribution consists of a portable C language library that can be included in users projects to build software that controls the core functionality of the MōVI Pro, such as gimbal pointing, lens motor control, camera start/stop, etc.

Freefly API v1.0 is supported on MōVI Pro GCU v1.2 or above through the COM1 or COM2 port, on MIMIC v1.2 or above through the COM1 port, and on MIMIC v1.3 or above through the COM1 or COM2 port.

Freefly's customer support team prioritizes end-user support and will not be able to answer API support requests. Please visit the Freefly Forum API Section for FAQs, additional resources, and bug reports.

QX Protocol

Freefly's custom communications protocol that provides this interface is called the "QX Protocol". This is a compact, lightweight binary messaging protocol that minimizes the size and complexity of messages, making it efficient for transmission on bandwidth constrained communications links.

Source code that allows building and parsing messages with this protocol is provided as a portable pure C language library that can be included in C/C++ projects directly, and used as a template in projects of other languages.

The QX protocol uses a client - server model where clients send read or write requests and receive current values, and servers send current values and receive read or write requests. In the Freefly API, the device connected to a COM port acts as the client and the MōVI Pro acts as the server.

Connectors and Signaling

The COM1/COM2 port on the MōVI Pro GCU and the COM1 port on the MIMIC are the specific ports enabled for API communications. The UARTs use 3.3V nominal signaling and are 5V tolerant. The connector needed to create a cable is a 6 pin "GH" series connector from JST, part number GHR-06V-S, crimp contacts part number SSHL-002T-P0.2. These are available on Digikey and other major distributors.

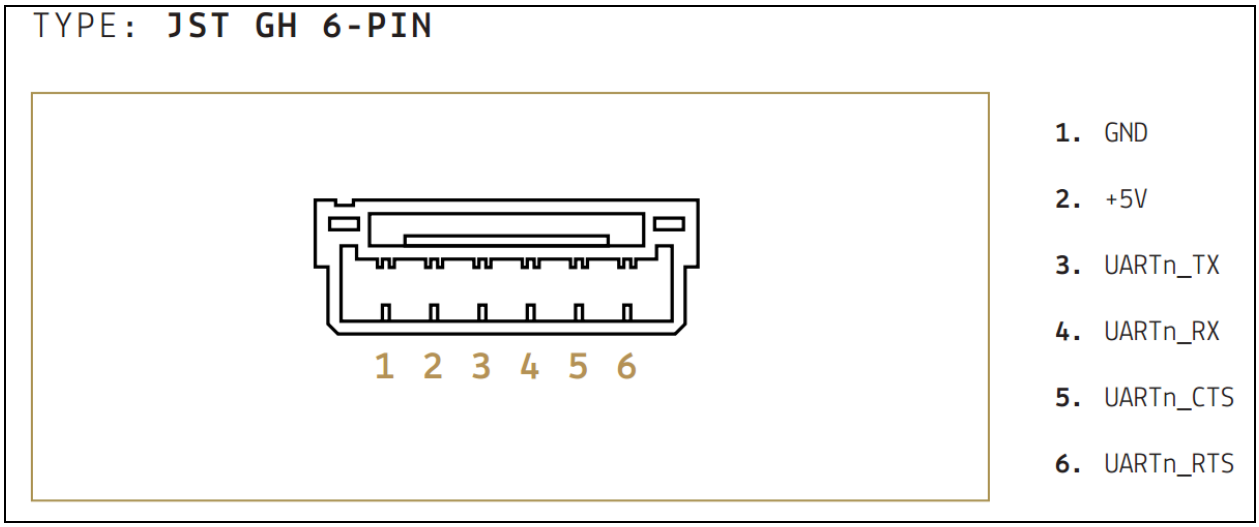
Freefly sells a cable (MōVI Pro COM to MōVI Controller Receiver Cable) that can be used to make custom cables more easily, as the 6 pin GH cable can be difficult to terminate by hand.

Freefly API v1.0 supported UART Serial Port Settings (111111 8-N-1):

Baud Rate	111,111bps (non-standard)
Data Bits	8

Parity	None
Stop Bits	1
Flow Control	Not used.

COM port connector pinout, looking into the GCU or MIMIC:



Example Applications

There are two examples included in the API distribution, one for Arduino and one for STM32 Nucleo development boards. They both use the core Freefly API library files to implement a basic controller using a commonly available Arduino header compatible joystick shield board. The most readily available one we have found is from Sparkfun.

Joystick board link - https://www.sparkfun.com/products/9760

Both of these examples essentially send a serial message periodically at 50 Hz to provide real time commands to the MōVI to move the gimbal, lens motors, and other common settings. They also receive a response message in return to each sent message.

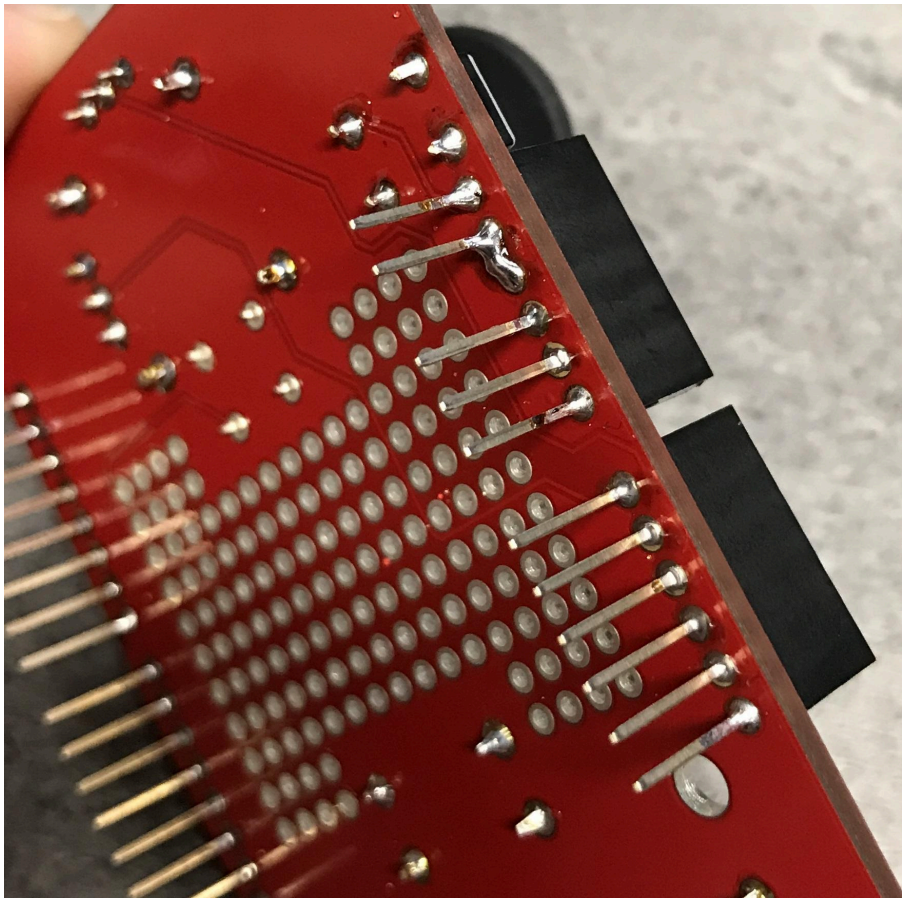
STM32

The STM32 Nucleo example uses an ST STM32F746 Nucleo development kit and a Sparkfun joystick shield board. The project provided is for the Keil V5 IDE, which can be downloaded for free for small sized projects.

STM32F746ZG Nucleo - http://www.st.com/en/evaluation-tools/nucleo-f746zg.html

One small modification is required for the sparkfun shield to be used with this board: The 5V pin on the shield is used as the source for the joystick analog reference, and thus must be cut and connected to 3.3V to match the analog reference of the STM32 nucleo board.

Joystick shield analog reference modification for STM32:



Arduino

The Arduino example uses an Arduino Mega 2560 or Uno development kit and a Sparkfun joystick shield board to form a basic embedded MōVI controller. The Mega is the best choice, as the memory is extremely limited on the Uno. The .ino extension file is the file that contains the primary application and acts as the “project file”.

The Mega and Uno use 5V as the analog reference. The reference voltage can be specified in the .ino file. Uncomment the associated definition in the .ino file to select your board.

The Due should also be usable with minimal modification to the code.

Arduino Mega 2560 - https://www.sparkfun.com/products/11061

Arduino Uno - https://www.sparkfun.com/products/11224

Code Overview

The following describes the primary library files included in the API distribution. There are two groups of

library related files:**QX Library Files**

These files require no modifications and should simply be included in the project.

- **QX_Protocol.c/.h** - This is the core library module that allows sending and receiving messages via Freeflys QX communications protocol.
- **QX_Parsing_Functions.c/.h** - Contains standard parsing macros for adding and removing values from a message buffer, while checking the range, applying scaling, etc.

QX Application Specific Files

These are files required by the QX library that are application specific. They contain application specific code that implements the customized and configurable portions of the communications protocol.

- **QX_Protocol_App.c/.h** - This module should contain application specific callback functions such as client and server parser callbacks, send message callback, wrapper functions for init, send, etc.
- **QX_API_Config.h** - Communication port enumeration type, quantity of clients and servers, etc.
- **FreeflyAPI.h** - Header that allows inclusion of the QX_library in C++ projects by including the QX_Protocol_App.h as extern.

Porting

The following describes the process of including and using the library in custom project. The examples are very helpful to use as templates for creating custom versions.

1. Copy the QX_Protocol and QX_Protocol_App headers into the project.
2. Determine the number of client and server role instances required for the project, and define these in the QX_API_Config.h file. For API uses, typically only a single client instance is used, but at least one server instance must be defined as well.
3. Define the QX_Comms_Port_e enumeration type to contain a list of the available ports for communications in QX_App_Config.h.
4. In QX_Protocol_App.c define a client parser callback function that will exchange data between messages and the application.

5. Call the QX_InitCli() function to initialize a client instance. The source address parameter should be QX_DEV_ID_MOVI_API_CONTROLLER.
6. Modify the commanded variable structure with command values. Call the send packet function to command the MōVI. QX_SendPacket_Cli_WriteABS(&QX_Clients[0], 277, QX_COMMS_PORT_UART, options);
7. Implement a custom callback send packet to comm port function that will send the constructed message to the communication port, or intermediate buffer.
8. Call the function QX_StreamRxCharSM() periodically in a loop to receive status messages from MōVI.

API Structures

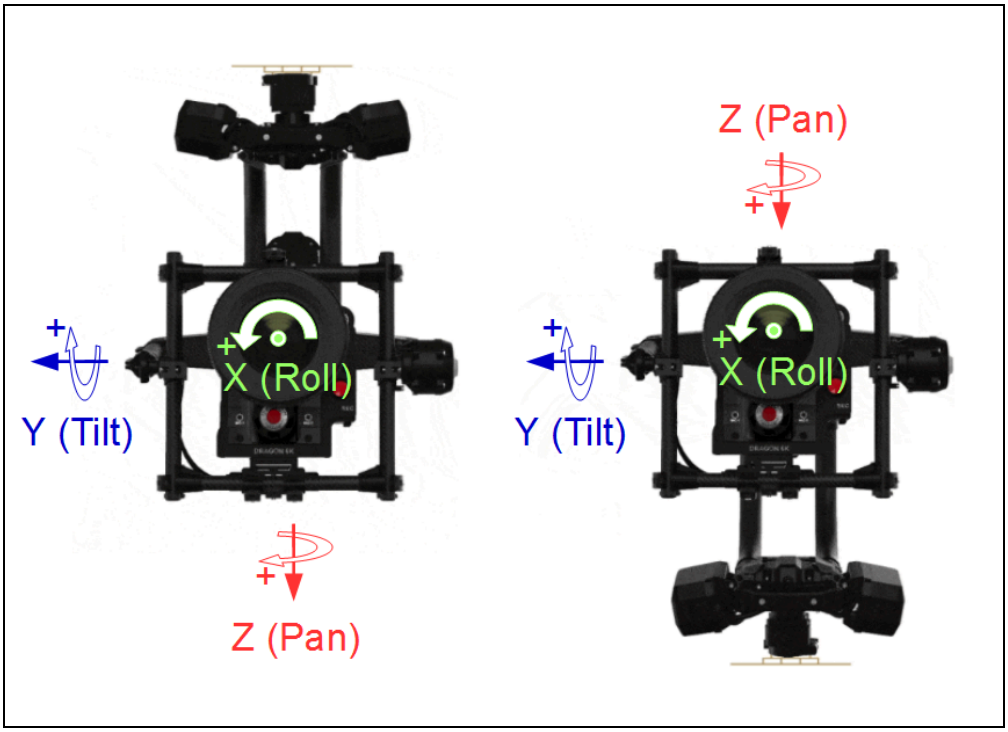
Control and status values are available in structures for use in application code:

> `FreeflyAPI.control` contains values used to control the gimbal, lens, and camera, scaled appropriately and transmitted to the COM port as a QX277 packet by `FreeflyAPI.send()`.
>
> `FreeflyAPI.status` contains values that convey gimbal, lens, and camera status, scaled appropriately after being received from the COM port as a QX287 packet.

See QBX_Protocol_App.h for the definition of these functions and structures. See example code for typical usage. See Appendix A for the on-the-wire definition of QX277 and QX287 packets.

Gimbal and Lens Control Overview

Gimbal control neutral pose axis definitions:



Gimbal Rate Control

To control the angular rate of a gimbal axis, set its control type to RATE. The input is given in rate of change of Euler angles applied in the order Pan, then Roll, then Tilt, with ±1.0 = ±(RC Rate Scale)°/s. RC Rate Scale is a setting configured with the MōVI Pro App.

Gimbal Absolute Angle Control

To control the absolute angle of a gimbal axis, set its control type to ABSOLUTE. The input can be given in Euler angles or as a quaternion:

> **Euler:** `gimbal_position_type_quaternions = 0`
> Euler angle control is applied in the order Pan, then Roll, then Tilt. Control values for each axis are scaled so that ±1.0 = ±180º. Roll and Tilt angle limits are applied as configured by the MōVI Pro App. The Quaternion R control value is ignored.
>
> **Quaternion:** `gimbal_position_type_quaternions = 1`
> Quaternion {r,i,j,k} control is applied with i=X, j=Y, and k=Z, corresponding to the neutral pose Roll, Tilt, and Pan axes, respectively. The values should have a vector magnitude $\sqrt{(r^2+i^2+j^2+k^2)} = 1.0$. For API forwarding through MIMIC, all three gimbal axes must have their control type set to ABSOLUTE. Roll and Tilt angle limits do not apply in Quaternion control mode.

When an absolute angle control command for the Pan axis is first received from a new input, an offset is applied to the Pan angle input such that the MōVI Pro's current Pan angle is maintained, to allow smooth transitioning between controllers. If an application requires absolute heading with respect to an external reference frame (e.g. for targeting), the gimbal orientation reported back in `FreeflyAPI.status` contains this information.

Although the gimbal does some interpolation and smoothing, absolute angle control is less tolerant of input timing variation than rate control. Extra care should be taken to ensure fast (**100Hz works well**), evenly-spaced inputs, especially if using MIMIC API forwarding.

Gimbal Kill

To disable torque on all three gimbal motors, set `gimbal_kill = 1`. To restore normal operation, reset `gimbal_kill = 0`. With multiple input sources, Kill control is prioritized similarly to other axis control inputs as described in **Input Priorities**.

Lens and Camera Control

To control the position of a lens motor, set its control type to ABSOLUTE. The input is given as ±1.0 = ±(Full Range). 0.0 corresponds to the center of the full range. Zoom may also be rate-controlled by setting its control type to RATE. The input is given as ±1.0 = ±(Full Zoom Rate).

Digital flags are also available in the `FreeflyAPI.control` structure for calibrating the lens motors, triggering camera run/stop, clearing faults, and setting subrange limits for lens axes. In general these flags are active high (value of 1) and are available to all controllers regardless of input priority. See example code for typical usage.

Deferring Controls

To defer control of a gimbal or lens axis to another controller, set its control type to DEFER. This is the default setting for all axes. A list of external controllers available and their priority of control is shown in **Input Priorities**.

Input Priorities

Gimbal and lens inputs from multiple sources are prioritized from highest to lowest as follows:

1. MIMIC inputs (MIMIC mode, COM1 Freefly API, Gestural FIZ, Gamepad, and COM2 Focus Knob).
    a. In most cases, multiple input sources can be used simultaneously with MIMIC:

MIMIC Mode	API Gimbal Control	API Lens Control
Kill	No	Yes
Off	Yes	Yes
MIMIC	No	Yes
Gamepad	Yes	Yes

2. MōVI Pro COM2 inputs (Freefly API, Spektrum, Futaba, MIMIC Beta, or MōVI Controller).
3. MōVI Pro COM1 inputs (Freefly API, Spektrum, Futaba, MIMIC Beta, or MōVI Controller).
4. App inputs.

Exceptions:

- MōVI Controller cannot control lens axes from COM2. This is to allow dual MōVI Controller operation: one for gimbal control on COM2 and one for lens control on COM1. This exception does not apply to API inputs.

Freefly API input sources can defer control of individual axes to lower-priority controllers by setting their control type to DEFER. Inputs will time out after a period of inactivity (typically 500ms). Timing out is equivalent to setting control type to DEFER on all axes. If no other external inputs are present, the MōVI Pro will return to Majestic mode for single operator control.

MIMIC API Forwarding

The MIMIC will forward API messages between its COM1 port and the MōVI Pro when it is on the same channel as and bound to the MōVI Pro. Axes that are deferred by the API input device on COM1 will take on their normal role based on the current MIMIC configuration. Replies from the MōVI Pro will be forwarded to the COM1 port in the form of QX287 packets, which the API parses into gimbal, lens, and camera status in `FreeflyAPI.status`. If no API inputs are received on COM1 for more than 500ms, the MIMIC will time out to normal operation.

Appendix A: On-the-Wire Definitions

The following contains definitions for API control and status packets as they would appear at the serial port (“on-the-wire”) if viewed with a terminal program or logic analyzer. In order to maximize throughput, API data is packaged in binary form and scaled to integer types where appropriate.

On-the-wire scaling guidelines:

On-the-Wire Type	API Structure Scaling	On-the-Wire Scaling
Signed 16-Bit Integer (Roll, Tilt, Pan, Quaternion)	-1.0f to 1.0f	-32767 to 32767
Unsigned 16-Bit Integer (Focus, Iris, Zoom)	-1.0f to 1.0f	0 to 65535

Axis definitions with on-the-wire types and ranges:

Axis ID	Axis Name	Control Signal Type	Control Signal Ranges
Gimbal RX	X, Roll or Quaternion I	Signed (int16_t)	**Rate:** ±32767 = ±(RC Rate Scale)°/s **Absolute:** ±32767 = ±180° **Quaternion:** ±32767 = ±1.0
Gimbal RY	Y, Tilt or Quaternion J	Signed (int16_t)	**Rate:** ±32767 = ±(RC Rate Scale)°/s **Absolute:** ±32767 = ±180° **Quaternion:** ±32767 = ±1.0
Gimbal RZ	Z, Pan or Quaternion K	Signed (int16_t)	**Rate:** ±32767 = ±(RC Rate Scale)°/s **Absolute:** ±32767 = ±180° **Quaternion:** ±32767 = ±1.0
Gimbal RR	Quaternion R	Signed (int16_t)	**Quaternion:** ±32767 = ±1.0
Lens LF	Focus	Unsigned (uint16_t)	**Rate:** N/A **Absolute:** 0 - 65535 = Full Range

Lens LI	Iris	Unsigned (uint16_t)	**Rate:** N/A **Absolute:** 0 - 65535 = Full Range
Lens LZ	Zoom	Signed (int16_t) / Unsigned (uint16_t)	**Rate:** ±32767 = ±(Full Rate) **Absolute:** 0 - 65535 = Full Range

Axis Control Flag on-the-wire values:

Control Flag [1:0] Value	Control Type
0 = 0b00	**Defer.** The transmitting controller requests that the receiving device ignore its control value for this axis. This can be used to give control of an axis to a lower priority controller. In the case of gimbal axes, if no other controller is present, the gimbal will be in single-operator (handle-based) Majestic Mode.
1 = 0b01	**Rate.** The transmitting controller requests that the receiving device interpret its control value for this axis as a rate command.
2 = 0b10	**Absolute.** The transmitting controller requests that the receiving device interpret its control value from this axis as a direct absolute displacement command.
3 = 0b11	**Absolute, Majestic.** The transmitting controller requests that the receiving device interpret its control value from this axis as an absolute displacement command, but with added user-configurable window and smoothing (Majestic Mode settings). This applies only to gimbal axes.

QX277 Control Packet on-the-wire definition:

Offset	QX Purpose	Value/Range	Data Function
0	HEADER ‘Q’	‘Q’ = 0x51	Header.
1	HEADER ‘X’	‘X’ = 0x58	Header.
2	LENGTH	28 = 0x1C	Number of bytes between LENGTH and CHECKSUM, exclusive.
3	ATTRIBUTE	0x95	Attribute, first byte.
4	ATTRIBUTE	0x02	Attribute, second byte.
5	OPTIONS	0x02	Reserved, must be set to 0x02 for API communication.

6	SOURCE_ADDR	0x0A	Reserved.
7	TARGET_ADDR	0x02	Reserved.
8	TRID	0x00	Reserved. (Transmit Request ID.)
9	RRID	0x00	Reserved. (Reply Request ID.)
10	PAYLOAD	0x00	Reserved. (Bind address and bind flags.)
11	PAYLOAD	0x00	Reserved. (Bind address and bind flags.)
12	PAYLOAD	0x00	Reserved. (Bind address and bind flags.)
13	PAYLOAD	0x00 - 0xFF	**Gimbal Control Flags** BIT 7: 0 = Euler, 1 = Quaternion BIT 6: 0 = Active, 1 = Kill BIT 5: RX Control Flag [1] BIT 4: RX Control Flag [0] BIT 3: RY Control Flag [1] BIT 2: RY Control Flag [0] BIT 1: RZ Control Flag [1] BIT 0: RZ Control Flag [0]
14	PAYLOAD	0x00 - 0xFF	Gimbal RX [15:8]
15	PAYLOAD	0x00 - 0xFF	Gimbal RX [7:0]
16	PAYLOAD	0x00 - 0xFF	Gimbal RY [15:8]
17	PAYLOAD	0x00 - 0xFF	Gimbal RY [7:0]
18	PAYLOAD	0x00 - 0xFF	Gimbal RZ [15:8]
19	PAYLOAD	0x00 - 0xFF	Gimbal RZ [7:0]
20	PAYLOAD	0x00 - 0xFF	Gimbal RR [15:8]
21	PAYLOAD	0x00 - 0xFF	Gimbal RR [7:0]
22	PAYLOAD	0x00 - 0x7F	**Lens Control Flags** BIT 7: Reserved

			BIT 6: Reserved BIT 5: LF Control Flag [1] BIT 4: LF Control Flag [0] BIT 3: LI Control Flag [1] BIT 2: LI Control Flag [0] BIT 1: LZ Control Flag [1] BIT 0: LZ Control Flag [0]
23	PAYLOAD	0x00 - 0xFF	Lens LF [15:8]
24	PAYLOAD	0x00 - 0xFF	Lens LF [7:0]
25	PAYLOAD	0x00 - 0xFF	Lens LI [15:8]
26	PAYLOAD	0x00 - 0xFF	Lens LI [7:0]
27	PAYLOAD	0x00 - 0xFF	Lens LZ [15:8]
28	PAYLOAD	0x00 - 0xFF	Lens LZ [7:0]
29	PAYLOAD	0x00 - 0xFF	**Lens Auxiliary Digital Byte 1:** **General Commands** BIT 7: Zoom Set Subrange BIT 6: Iris Set Subrange BIT 5: Focus Set Subrange BIT 4: Record Button State BIT 3: Set Button State BIT 2: Menu Button State BIT 1: Start Autocal, All Axes BIT 0: Clear All Faults
30	PAYLOAD	0x00 - 0x7F	**Lens Auxiliary Digital Byte 2:** **LANC or Sony Multi Terminal Control** BIT 7: Reserved BIT 6: Shutter BIT 5: Auto Iris

			BIT 4: Auto Focus BIT 3: Iris Minus BIT 2: Iris Plus BIT 1: Focus Minus BIT 0: Focus Plus
31	CHECKSUM	0x00 - 0xFF	QX Packet Checksum 255 - [SUM(Byte 3 thru Byte 28) Mod 256]

QX287 Status Packet on-the-wire definition:

Offset	QBX Purpose	Value/Range	Data Function
0	HEADER 'Q'	'Q' = 0x51	Header.
1	HEADER 'X'	'X' = 0x58	Header.
2	LENGTH	33 = 0x31	Number of bytes between LENGTH and CHECKSUM, exclusive.
3	ATTRIBUTE	0x9F	Attribute, first byte.
4	ATTRIBUTE	0x02	Attribute, second byte.
5	OPTIONS	0x40	Reserved.
6	SOURCE_ADDR	0x02	Reserved.
7	TARGET_ADDR	0x02	Reserved.
8	TRID	0x00	Reserved. (Transmit Request ID.)
9	RRID	0x00	Reserved. (Reply Request ID.)
10	PAYLOAD	0x00	Reserved. (Bind address and bind flags.)
11	PAYLOAD	0x00	Reserved. (Bind address and bind flags.)
12	PAYLOAD	0x00	Reserved. (Bind address and bind flags.)
13	PAYLOAD	0x00 - 0xFF	Battery A Voltage 10.0 + (Byte Value * 0.1) [V]
14	PAYLOAD	0x00 - 0xFF	Battery B Voltage

			10.0 + (Byte Value * 0.1) [V]
15	PAYLOAD	0x00 - 0xFF	**Gimbal Status Byte 1** BIT 7: Reserved BIT 6: IMU Error BIT 5: Reserved BIT 4: Drive Error / Drives Disabled BIT [3:0]: Reserved
16	PAYLOAD	0x00 - 0xFF	**Gimbal Status Byte 2** BIT [7:5]: Reserved BIT 4: Gimbal Error BIT 3: GPS Locked BIT 2: Radio L.O.S. BIT 1: GPS L.O.S. BIT 0: Compass Error
17	PAYLOAD	0x00 - 0xFF	Gimbal RR[15:8]
18	PAYLOAD	0x00 - 0xFF	Gimbal RR[7:0]
19	PAYLOAD	0x00 - 0xFF	Gimbal RX[15:8]
20	PAYLOAD	0x00 - 0xFF	Gimbal RX[7:0]
21	PAYLOAD	0x00 - 0xFF	Gimbal RY[15:8]
22	PAYLOAD	0x00 - 0xFF	Gimbal RY[7:0]
23	PAYLOAD	0x00 - 0xFF	Gimbal RZ[15:8]
24	PAYLOAD	0x00 - 0xFF	Gimbal RZ[7:0]
25	PAYLOAD	0x00 - 0xFF	Radio Slot 1 Packet Rx/s
26	PAYLOAD	0x00 - 0xFF	Radio Slot 2 Packet Rx/s
27	PAYLOAD	0x00 - 0xFF	**FIZ General Status** BIT 7-3: Reserved BIT 2: Zoom Range Limits Active

			BIT 1: Iris Range Limits Active BIT 0: Focus Range Limits Active
28	PAYLOAD	0x00 - 0xFF	**FIZ Camera Status** BIT 7-1: Reserved BIT 0: Camera Record Status
29	PAYLOAD	0x00 - 0xFF	Focus Axis State
30	PAYLOAD	0x00 - 0xFF	Iris Axis State
31	PAYLOAD	0x00 - 0xFF	Zoom Axis State
32	PAYLOAD	0x00 - 0xFF	Focus Axis Position [15:8]
33	PAYLOAD	0x00 - 0xFF	Focus Axis Position [7:0]
34	PAYLOAD	0x00 - 0xFF	Iris Axis Position [15:8]
35	PAYLOAD	0x00 - 0xFF	Iris Axis Position [7:0]
36	PAYLOAD	0x00 - 0xFF	Zoom Axis Position [15:8]
37	PAYLOAD	0x00 - 0xFF	Zoom Axis Position [7:0]
38	CHECKSUM	0x00 - 0xFF	QX Packet Checksum 255 - [SUM(Byte 3 thru Byte 35) Mod 256]